Software Architecture- 02

KALINGA GUNAWARDHANA

DEPT OF COMPUTING & INFORMATION SYSTEMS

SABARAGAMUWA UNIVERSITY OF SRI LANKA

Component Diagrams

UML component diagrams are concerned with the components of a system. **Components** are the independent, encapsulated units within a system.

Each component provides an interface for other components to interact with it.

Component diagrams are used to visualize how a system's pieces interact and what relationships they have among them.

Difference…

Component diagrams are different from most other diagrams

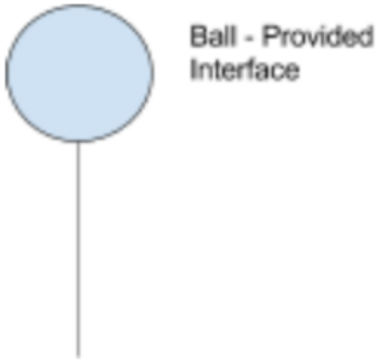
They show high-level structure and not details like attributes and methods.

They are purely focused on components and their interactions with each other.

Ball connectors

- Component diagrams have **ball connectors**, which represent a provided interface.

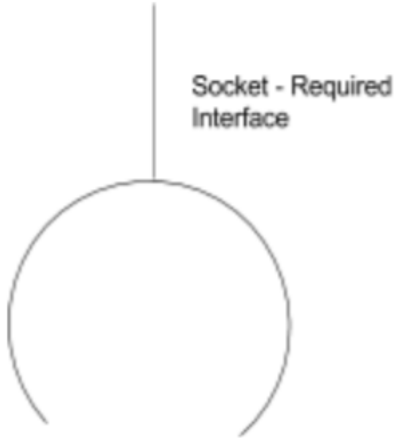
- A provided interface shows that a component offers an interface for others to interact with it.



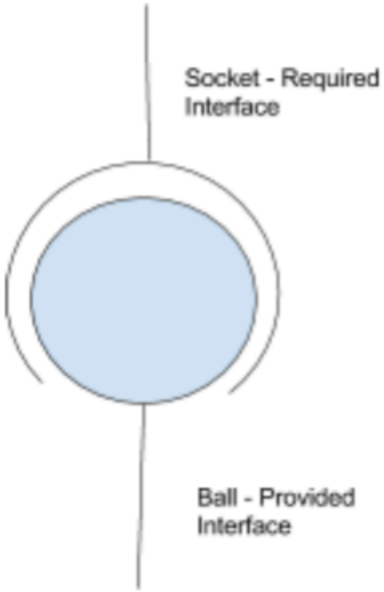
Socket connectors

Component diagrams also have **socket connectors** that display a required interface.

To show that a component expects a certain interface. This required interface is to be satisfied or provided by some other component.



Finally, component diagrams can illustrate an assembly relationship. An assembly relationship occurs when one component's provided interface matches another component's required interface.



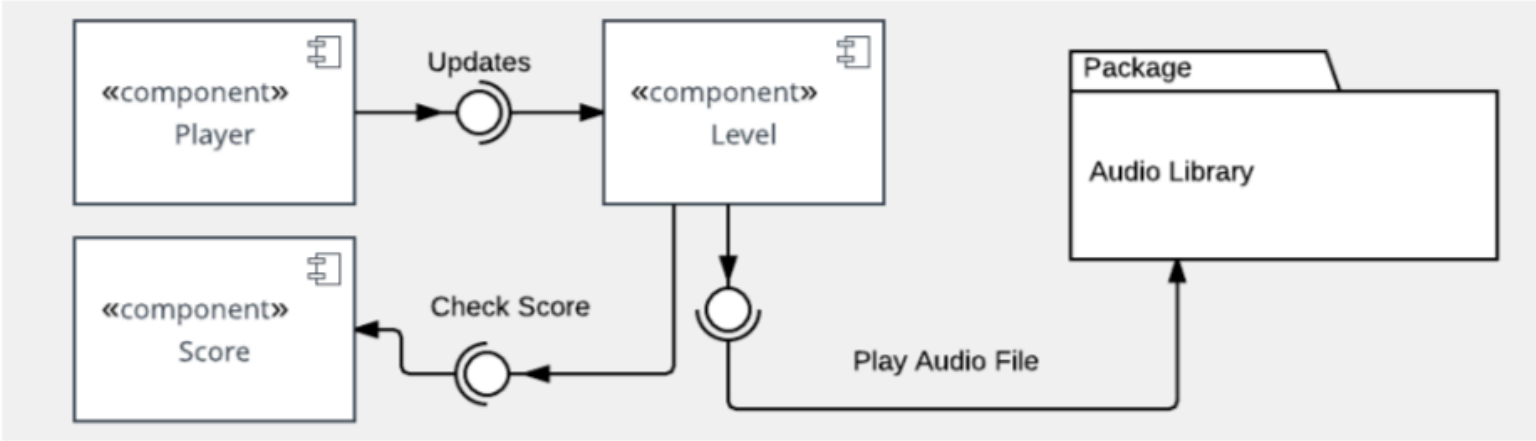
How to build a component diagram?.

-You must identify the main objects used in the system.

- Next, the relevant libraries for the system need to be identified

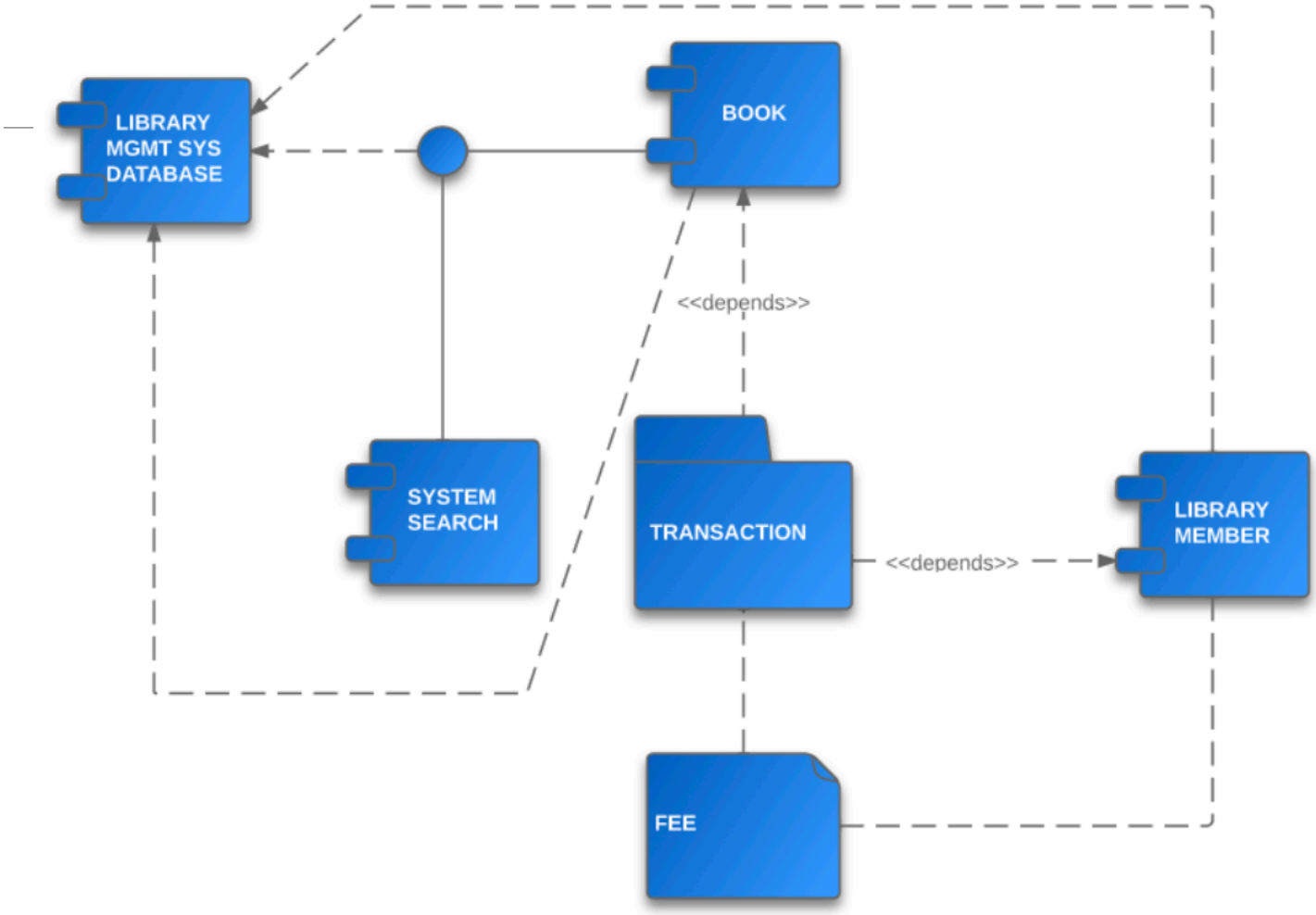
- Finally , the relationship between these components would need to be identified.

Example


«component»
Player
Updates
«component»
Level
Package
Audio Library
«component»
Score
Check Score
Play Audio File

Exercise

Create a component diagram for a library system.


LIBRARY
MGMT SYS
DATABASE
BOOK
<<depends>>
SYSTEM
SEARCH
TRANSACTION
<<depends>>
LIBRARY
MEMBER
FEE

Package Diagrams

A **package** groups together elements of software that are related.

Elements can be related based on data, classes, or user tasks.

A package can also define a “namespace” for elements it contains, that is, a package is named and can organize the named elements of software into a separate scope.

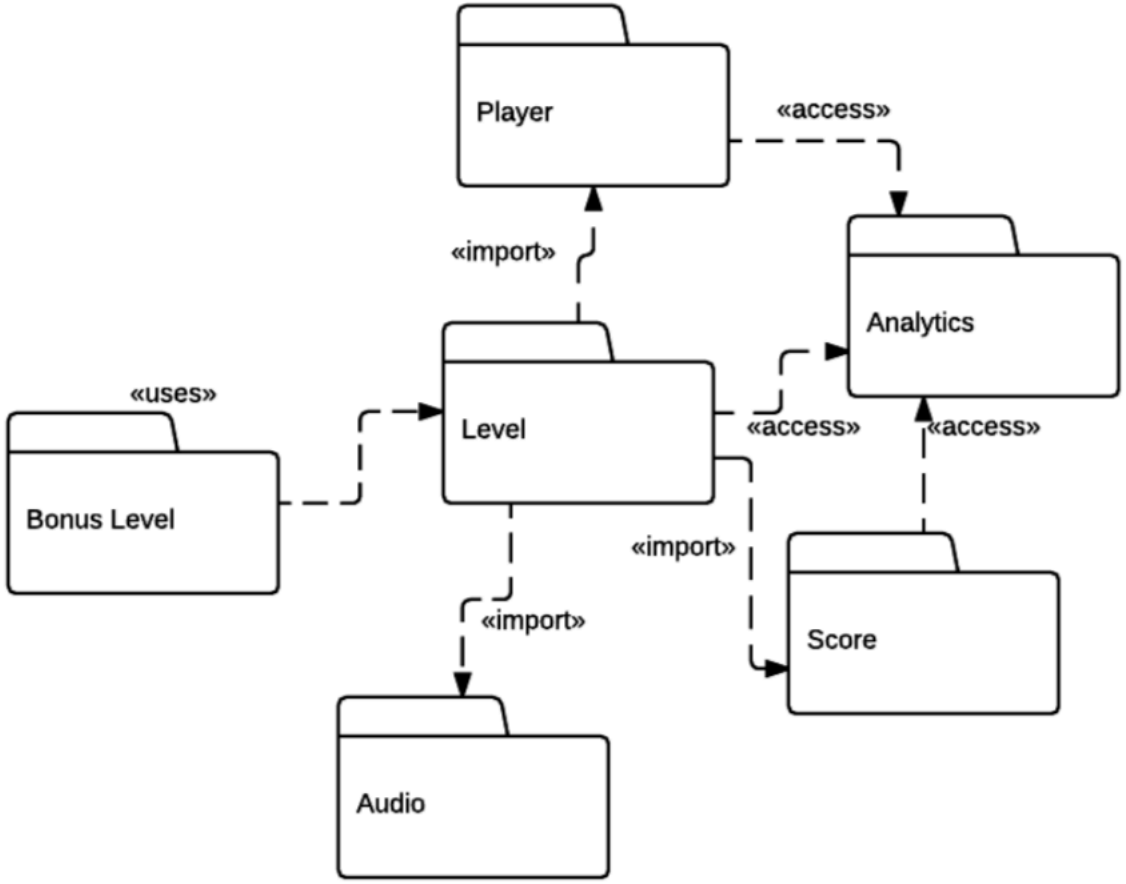
Cont…

Package diagrams show packages and the dependencies between them.

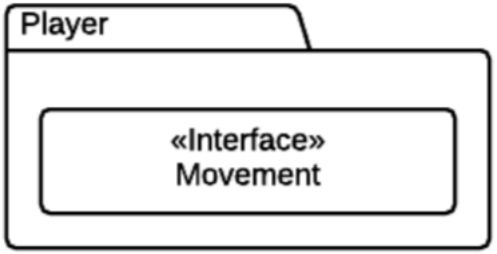
These diagrams can organize a completed system into packages of related **packageable elements**, which could include data, classes, or even other packages.

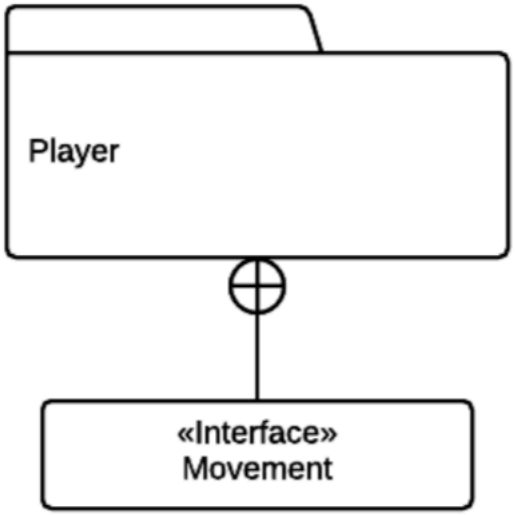
Package diagrams help provide high-level groupings of a system so that it is easy to see how a package contains related elements as well as how different packages depend on each other

Example


Player
«access»
«import»
Analytics
«uses»
Level
«access»
«access»
Bonus Level
«import»
«import»
Score
Audio

Notations



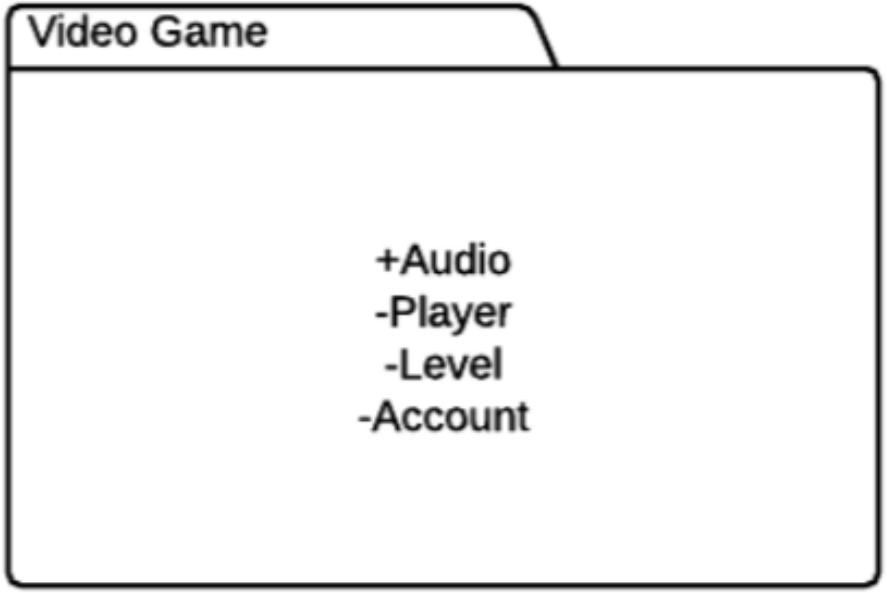


Note…

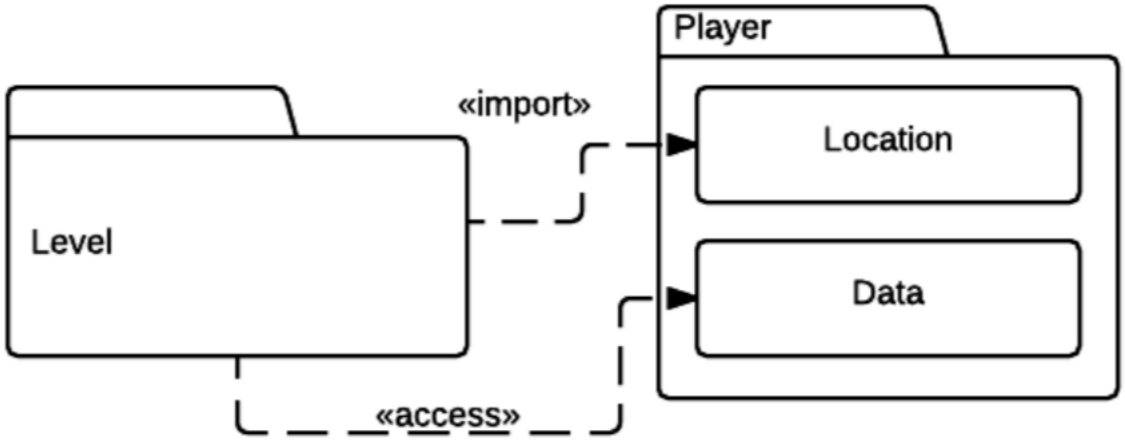
Contained elements can also be listed in a package by their names.

These names can be partially qualified, but they should be unique within the package.

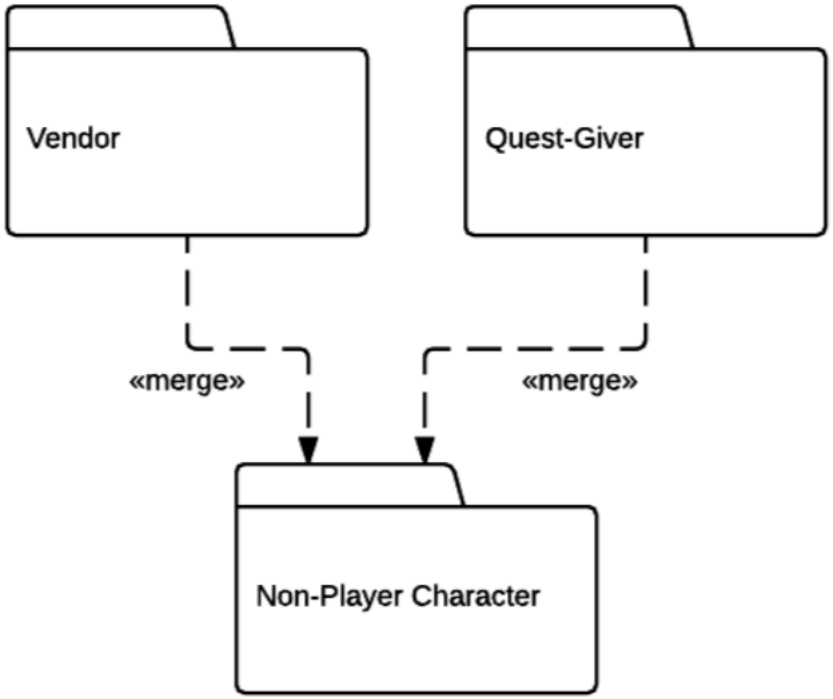
Public and Private



A package can import an element from another package, as in the example below where the Level package is importing Location from the Player package.



Packages can also be merged, as in the diagram below. Merging typically occurs when two packages or concepts need to come together into one. This is a use of generalization that allows different definitions to be provided for the same concept.



Exercise

Create a package diagram for the human society.

Deployment Diagram

UML deployment diagrams are used to visualize the deployment details of a software system.

The diagrams include more than just code, but also separate libraries, an installer, configuration files, and many other pieces.

The deployment environment, or deployment target, can be very specific and involve particular hardware devices.

Artifacts

Deployment diagrams deal with **artifacts**. Artifacts are a physical result of the development process.

Artifacts for a video game might include things like an executable to run the game, an installer to install the game, audio libraries for sound, and multimedia assets.

These are created as outcomes of producing the system and are the final pieces to be put together.

There are two types of deployment diagrams:

- Specification-level diagrams
- Instance level diagrams

Specification level diagram

A specification-level diagram provides an overview of artifacts and deployment targets, without referencing specific details like machine names.

It focuses on a general overview of your deployment rather than the specifics.

Instance level diagram

An instance-level diagram is a more specific approach that maps specific artifacts to specific deployment targets.

They can identify specific machines and hardware devices.

This approach is usually used to highlight the differences in deployments among development, staging, and release builds.

Nodes

When creating deployment diagrams, it is important to use the correct notation for the various elements.

One of the most important aspects of the deployment diagram is the node.

Nodes are deployment targets that contain artifacts available for execution.

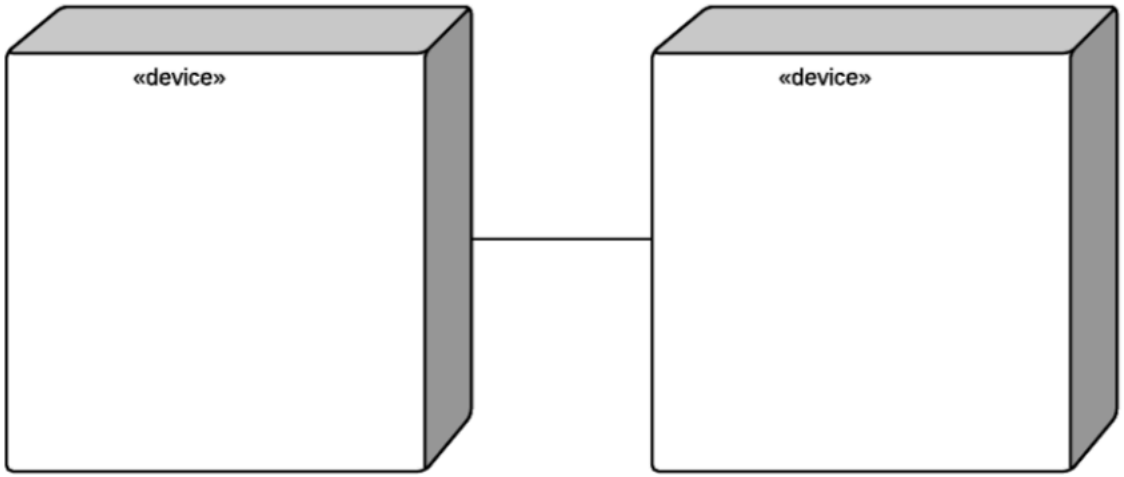
In a deployment diagram, they look like 3D boxes.

Hardware devices are also displayed as 3D boxes, only they have a “device” tag on them to differentiate them.

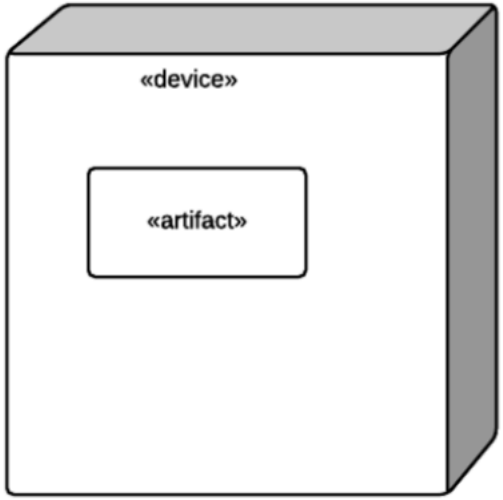
Relationships

Relationships can be represented a few different ways in deployment diagrams.

A solid line between two nodes shows a relationship between deployment targets. The line shows that the two nodes have a communication path between them. This relationship typically identifies a particular communication protocol.

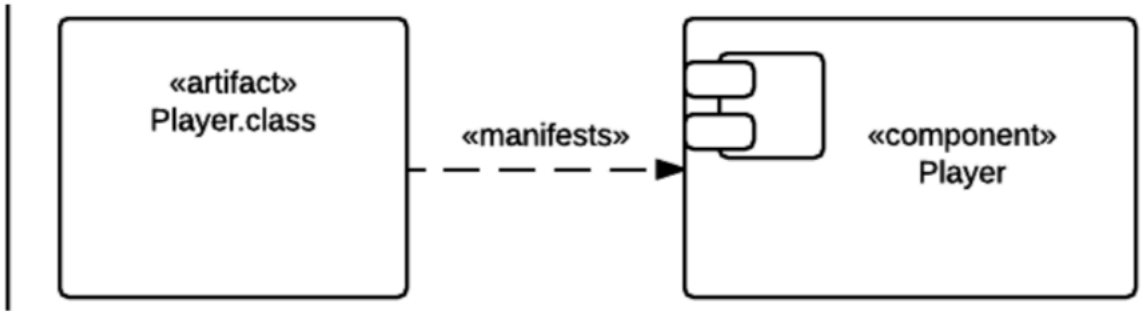

«device»
«device»

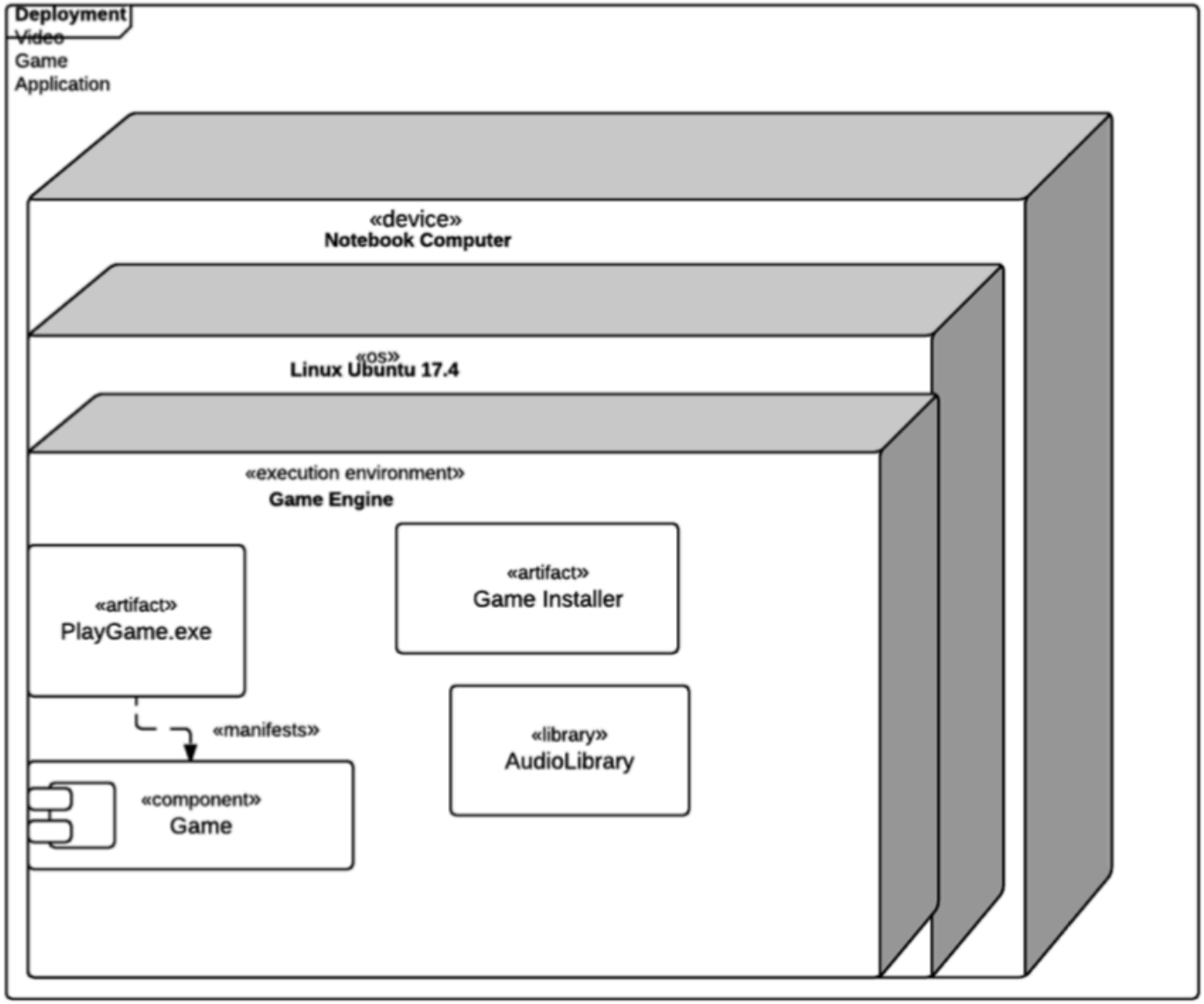
If an artifact is drawn inside a node box, this shows that an artifact is deployed to a node. This also means that the artifact cannot function without this deployment target.



Manifestation

Manifestation is a relationship where an artifact is a physical realization of a software component. It can be represented with a “manifests” indicator, as below.




Deployment
Video
Game
Application
«device»
Notebook Computer
«OS»
Linux Ubuntu 17.4
«execution environment»
Game Engine
«artifact»
PlayGame.exe
«artifact»
Game Installer
«manifests»
«library»
AudioLibrary
«component»
Game

Notes

In a deployment diagram, there is a distinct hierarchy of deployment targets.

In this example, the application, the device, the operating system, and the execution environment are all listed. When you look at each of these nodes, it is clear what environment is necessary to run the application.

Note that only the most important artifacts are represented in this diagram so that it is kept at a high level.

Deployment diagrams help provide consistency and organization to deployments, which helps avoid system failures. The diagrams also help keep track of the files and executables needed to deploy and run the software.